PS-01 Computer Vision

Vardaan Saxena

October 2026

Gate 1

How does a computer see?

Question 1: What is a digital image? Explain a pixel, image resolution, and the matrix representation of an image.
A **digital image** is a representation of a visual scene using a finite collection of discrete picture elements called pixels. Unlike a continuous scene in the physical world, a digital image stores visual information at discrete spatial locations, with each location containing a numerical value representing its intensity or colour.
A digital camera captures light from the surroundings and converts it into digital data. The resulting image can be stored, displayed, analysed, and processed by a computer. For an autonomous vehicle, this image may contain information about road lanes, vehicles, pedestrians, traffic signs, and obstacles.
A pixel, short for picture element, is the smallest individual sampling unit of a digital image. Each pixel represents the visual information associated with a particular location in the image.
In a grayscale image, a pixel typically stores one intensity value, commonly between 0 and 255 for an 8-bit image. Here, 0 represents black, 255 represents white, and intermediate values represent shades of gray. - In an RGB colour image, each pixel stores three intensity values corresponding to red, green, and blue. Each channel commonly ranges from 0 to 255 in an 8-bit-per-channel image. For example, an RGB pixel represented by $(255, 0, 0)$ corresponds to pure red, while $(0, 0, 0)$ corresponds to black. A pixel does not necessarily represent a distinct physical object. Instead, it records sampled visual information at a particular location. A road marking, for instance, may occupy hundreds or thousands of pixels.
Image resolution describes the dimensions of a digital image in pixels, usually expressed as width $\times$ height. For example, an image with a resolution of 640×480 contains 640 pixels horizontally and 480 pixels vertically. Its total number of pixels is

$$N = 640 \times 480 = 307{,}200$$

Higher-resolution images generally preserve finer spatial details because they contain more samples of the scene. However, they also tend to require more

storage and computational resources to process. It is important to distinguish resolution from pixel intensity: resolution tells us how many pixels make up the image, whereas intensity describes the visual value stored at each pixel. Increasing resolution does not automatically guarantee a better image, since lighting, focus, noise, and other factors also affect image quality.

A grayscale image can be represented as a **two-dimensional matrix** I of size $H \times W$, where H is the height and W is the width of the image. Each element $I(i, j)$ represents the intensity of the pixel at row i and column j.

For an 8-bit grayscale image,

$$I(i, j) \in \{0, 1, 2, \ldots, 255\}.$$

For example, a small grayscale image may be represented as

$$I = \begin{bmatrix} 0 & 50 & 100 \\ 120 & 180 & 220 \\ 255 & 200 & 80 \end{bmatrix}.$$

An RGB image consists of three channels and can be represented as a three-dimensional array of size $H \times W \times 3$. The pixel at location (i, j) is given by

$$I(i, j) = [R(i, j), G(i, j), B(i, j)],$$

where R, G, and B denote the red, green, and blue intensity values, respectively.

This matrix representation enables computers to perform image processing operations such as filtering, thresholding, and edge detection. In autonomous vehicles, these operations help identify road markings and estimate lane boundaries.

Question 2: What are RGB and grayscale images? Why might an autonomous vehicle process them differently?

RGB images

An RGB image is a colour image that represents each pixel using three colour channels: Red (R), Green (G), and Blue (B). By combining different intensities of these three primary colours, a wide range of colours can be represented.
In a typical 8-bit-per-channel image, each channel has an intensity value between 0 and 255. Therefore, a pixel can be represented as a triplet (R, G, B). For example, $(255, 0, 0)$ represents red, $(0, 255, 0)$ represents green, and $(0, 0, 255)$ represents blue. An RGB image of height H and width W can be represented as a three-dimensional array of dimensions $H \times W \times 3$. Since each pixel stores three values, RGB images contain more data than grayscale images of the same dimensions.

Grayscale images

A grayscale image represents visual information using different shades of gray, ranging from black to white, without explicitly storing colour information. In a typical 8-bit grayscale image, each pixel contains a single intensity value between 0 and 255. A value of 0 represents black, 255 represents white, and intermediate values represent different shades of gray. A grayscale image of height H and width W is represented as a two-dimensional array of dimensions $H \times W$. Since each pixel stores only one intensity value, grayscale images generally require less memory and less computational effort to process than RGB images of the same dimensions.
An autonomous vehicle may use RGB and grayscale images for different tasks depending on the type of visual information required.

- RGB images preserve colour information. This is useful for distinguishing traffic-light states, identifying coloured traffic signs, and separating objects based partly on their colour.

- Grayscale images emphasise intensity variations. They are often useful for detecting edges, identifying lane markings, and analysing changes in brightness between neighbouring regions.

- Grayscale processing can be computationally cheaper. Since it operates on one channel instead of three, it reduces the amount of image data that certain algorithms need to process. This can be advantageous for real-time applications.

- The choice depends on the task. Grayscale conversion discards explicit colour information, so it may make some objects harder to distinguish. RGB processing retains that information but can require more computation. Neither representation is universally better.

For example, an autonomous vehicle might use an RGB image to recognise a red traffic light, while using a grayscale version of the same image to detect the edges of lane markings. In practice, the vehicle's perception system may use several representations of the same camera image at different stages of processing.

Gate 2

Basic Image Processing

Question 1: What is Gaussian Blur and why is it useful before detecting features?
Gaussian blur is an image-processing technique used to smooth an image and reduce noise. It replaces each pixel's value with a weighted average of the values of its neighbouring pixels. The weights are determined by a Gaussian function, which assigns greater importance to nearby pixels.

The two-dimensional Gaussian function is given by

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}},$$

where x and y represent positions relative to the centre of the filter, and σ controls the spread of the Gaussian distribution. A larger value of σ generally produces stronger blurring.
The blur is applied by moving a small matrix called a kernel across the image. At each position, the kernel's weights are multiplied by the corresponding pixel values, and the results are added to calculate the new pixel value. This operation is called ***convolution***.
Gaussian blur helps in several ways:

- Noise reduction: It smooths small, irregular intensity variations that may otherwise be mistaken for meaningful features.
- Improved edge detection: Algorithms such as Canny edge detection identify sudden changes in intensity. Reducing noise beforehand can prevent these algorithms from detecting too many false edges.
- More reliable feature detection: Smoothing the image can make important structures, such as lane markings and road boundaries, easier to identify without interference from minor variations.
- Reduced false detections: It can suppress tiny details caused by noise or surface texture that are irrelevant to the task.

Question 2: What is HSV colour space? Why can it sometimes be more useful than RGB for road/lane detection?
HSV is a colour representation model that describes colours using three components: Hue (H), Saturation (S), and Value (V). Unlike RGB, which represents a colour through the intensities of red, green, and blue, HSV separates colour

information from its saturation and brightness.
- Hue (H): Represents the basic colour, such as red, yellow, green, or blue. It is commonly expressed as an angle around a colour wheel.
- Saturation (S): Represents the intensity or purity of a colour. High saturation indicates a vivid colour, while low saturation indicates a more washed-out or grayish colour.
- Value (V): Represents brightness. Higher values correspond to brighter colours, while lower values correspond to darker colours.
An RGB image stores three colour-channel values per pixel. Similarly, an HSV image stores three values per pixel, but these values describe different properties of the colour. HSV is a different representation of the same underlying colour information, not additional information.
In road images, lane markings may be white or yellow, while the road surface is often gray. HSV can make it easier to distinguish these markings because it separates colour characteristics from brightness.

- Colour-based filtering: Yellow lane markings can be isolated by selecting an appropriate range of Hue and Saturation values. This can help separate them from a gray road surface.

- Handling brightness variations: Since Hue and Saturation are represented separately from Value, colour-based filtering can sometimes remain more effective when brightness changes. However, Hue becomes unreliable for very dark or nearly gray pixels, and shadows can still affect detection.

- Detecting white lane markings: White markings generally have high Value and low Saturation. These properties can be used to distinguish them from darker road surfaces.

- Simpler thresholding: HSV allows the algorithm to select pixels based on particular colour and brightness ranges, which can be useful when constructing a mask containing potential lane markings.

Question 3: What does Canny edge detection do?
Canny edge detection is an image-processing algorithm used to identify significant edges in an image. An edge is a boundary between regions where pixel intensity changes sharply, such as the boundary between a white lane marking and a darker road surface. The Canny algorithm produces a binary edge image in which detected edges are typically represented by white pixels and non-edge regions by black pixels. It is widely used in computer vision because it aims to detect meaningful edges while reducing noise and avoiding unnecessary or duplicate edge responses.

The Canny algorithm involves five main stages:
1. Noise reduction: Gaussian blur is applied to smooth the image and reduce noise that might produce false edges.
2. Gradient calculation: The algorithm measures changes in intensity in horizontal and vertical directions to estimate the strength and direction of edges.
3. Non-maximum suppression: The algorithm thins detected edges by retaining local maxima in the gradient direction.
4. Double thresholding: Two thresholds classify pixels as strong edges, weak edges, or non-edge pixels according to their gradient magnitudes.
5. Edge tracking by hysteresis: Weak edges connected to strong edges are retained, while isolated weak edges are discarded.

Question 4: What is a Region of Interest (ROI), and why can it reduce false detections?
A Region of Interest (ROI) is a selected portion of an image on which an image-processing algorithm focuses its analysis. Instead of processing every pixel in the entire image, the algorithm considers only the region that is relevant to the task. In autonomous driving, the ROI for lane detection is typically selected from the lower portion of the camera image, where the road surface and nearby lane markings are most likely to appear. Regions such as the sky, distant buildings, and roadside scenery may be excluded because they are generally irrelevant to detecting the immediate lane boundaries. An ROI can be defined using a rectangular region, a polygon, or another suitable shape. For lane detection, a polygon covering the visible road is often more useful than a rectangle because roads commonly occupy a roughly trapezoidal region in a forward-facing camera image.
Why does an ROI reduce false detections?
An image may contain many edges that are unrelated to lane markings, including building edges, trees, road signs, shadows, and vehicles. If an algorithm processes the entire image, these features may be incorrectly identified as potential lane boundaries.
Restricting processing to an appropriate ROI helps in several ways:

- Eliminates irrelevant regions: Edges outside the selected road region are ignored.
- Reduces false positives: Fewer irrelevant features are considered as possible lane markings.
- Reduces computational effort: Processing fewer pixels can make the pipeline more efficient.
- Improves lane detection: The algorithm can focus on features within the

likely drivable region.

However, an ROI does not guarantee correct detection. If the selected region excludes part of a lane marking, or includes distracting features such as shadows and road cracks, errors can still occur. The ROI must therefore be selected appropriately for the camera view and road conditions.

Gate 3

Detecting Structure

Question 1: What is a Contour? What Information Can a Contour Provide?

A contour is a curve that joins a sequence of points along the boundary of an object or a connected region in an image. In computer vision, contours are commonly extracted from binary images, where objects of interest are separated from the background. For example, in a binary image of a road, a contour may trace the boundary of a lane marking, a vehicle, or another visible region. Contours help represent the shapes of objects without requiring the algorithm to analyse every pixel individually. In OpenCV, contours are commonly extracted using the cv2.findContours() function, typically after converting an image into a suitable binary representation

Information Provided by Contours:

- **Position:** Contour coordinates indicate the location of an object's boundary in the image.
- **Shape and geometry:** Contours can be used to estimate an object's shape, area, and perimeter.
- **Bounding box:** A bounding rectangle describes the approximate spatial extent of a region.
- **Centroid:** The centroid provides an estimate of the geometric centre of a region.
- **Shape comparison:** Contours can be compared to determine similarities between shapes.
- **Hierarchy:** Contour hierarchy represents relationships between nested boundaries, such as the outer boundary of a ring and its inner boundary.

Application in Autonomous Vehicles:

Contours can help analyse lane markings, road boundaries, and other objects in an image. For example, after thresholding an image to isolate bright lane markings, contours can be extracted to examine their shapes, areas, and positions.

However, contours describe boundaries rather than identifying objects by themselves. Additional processing is required to distinguish relevant road features from shadows, road texture, and other irrelevant regions.

Question 2: What is the Hough Transform? How can it be used to find lane lines?

The Hough Transform is a feature-detection technique used in computer vision to identify geometric shapes, particularly straight lines, in an image. It works by transforming image points into a parameter space, where points that belong to

the same line contribute votes toward a common set of line parameters. In lane detection, an edge-detection algorithm such as Canny first identifies potential edges in a road image. The Hough Transform then analyses these edge pixels to identify groups of points that may lie along straight lane boundaries.A common representation of a straight line is

$$y = mx + c$$

where m is the slope and c is the vertical intercept. However, this representation cannot describe vertical lines conveniently. Therefore, the Hough Transform commonly uses the polar representation

$$\rho = x\cos\theta + y\sin\theta$$

where: - x, y are the coordinates of an edge pixel. - ρ is the perpendicular distance from the origin to the line. - θ is the angle of the line's normal relative to the horizontal axis. Each edge pixel can correspond to multiple possible lines. In parameter space, these possibilities form a curve. When several edge pixels belong to the same straight line, their curves intersect near a common (ρ, θ) pair. The algorithm identifies these intersections using an accumulator, which records votes for candidate lines.
How is it used to find lane lines?
A typical lane-detection pipeline using the Hough Transform follows these steps:

- Input road image A camera image containing visible lane markings
- Preprocessing and edge detection Gaussian blur reduces noise; Canny extracts potential edges
- Region of Interest (ROI) Restricts analysis to the relevant road region
- Hough Transform Votes for candidate straight lines using edge pixels
- Lane-line estimation Selects plausible lines corresponding to lane boundaries

OpenCV provides two commonly used methods:
- cv2.HoughLines() implements the standard Hough Transform and returns lines in terms of ρ and θ.
- cv2.HoughLinesP() implements the probabilistic Hough Transform and returns line segments with endpoints, making it convenient for many lane-detection applications.

Gate 4

Lane/Path Perception

Question 1: Why is lane/path detection useful for an autonomous ground vehicle?
Lane and path detection allow an autonomous ground vehicle to understand where it should travel and how it is positioned relative to the road. By analysing camera images, the vehicle can identify lane markings, estimate road boundaries, and determine a suitable region in which to drive. Lane detection and path detection are related but distinct concepts. Lane detection identifies lane markings and boundaries, while path detection or estimation determines a route or drivable path that the vehicle can follow. Both contribute to the vehicle's perception of its driving environment.

Question 2: Why can shadows, road texture, poor lighting, missing markings, or clutter create false detections?
Lane detection algorithms analyse visual features such as colour, brightness, edges, and shapes to identify lane boundaries. However, these features are not unique to lane markings. Other elements in a road image can produce similar visual patterns, causing the algorithm to identify features that are not actual lane boundaries. These incorrect identifications are known as false detections.
Common causes of false detections:
Shadows: Trees, buildings, and other vehicles can cast shadows across the road. The resulting changes in brightness may produce strong edges that resemble lane markings.

Road texture: Cracks, patches, repaired surfaces, and textured asphalt can create numerous edges and irregular patterns. Edge-detection algorithms may mistake some of these features for lane boundaries.

Poor lighting: Low light, glare, overexposure, and sudden changes in illumination can reduce the contrast between lane markings and the road surface. This makes genuine markings harder to detect and may introduce misleading features.

Missing or faded markings: Worn-out, partially obscured, or absent lane markings provide incomplete visual information. The algorithm may fail to detect the actual lane boundaries or incorrectly infer them from nearby features.

Clutter: Vehicles, pedestrians, road signs, barriers, and roadside objects introduce additional edges and shapes. These can be mistaken for lane boundaries, particularly when they overlap with the road region.

Question 3: Why is lane detection alone not sufficient for a fully autonomous vehicle?
Lane detection helps an autonomous vehicle identify lane markings and estimate its position on the road. However, knowing where the lane boundaries are does not tell the vehicle everything it needs to know to drive safely. A fully autonomous vehicle must perceive its surroundings, make decisions, plan

its movements, and control its motion.
Key limitations of Lane detection:

- *Obstacle Detection:* Lane detection alone cannot reliably identify and avoid obstacles such as pedestrians, objects or debris lying on the road.

- *Traffic rules and signals:* Certain signals which are not obstacles must also be understood. A vehicle must recognise traffic lights and etc.

- *Changing road conditions:*Lane markings may be faded, missing, obscured by snow, or absent on unmarked roads. The vehicle must still navigate safely in these situations.

- *Motion Planning:*Identifying lane boundaries does not determine the safest route or whether the vehicle should turn, change lanes, stop, or overtake.

Gate 5

Perspective and BEV

Question 1: What is perspective distortion?
Perspective distortion refers to the apparent change in the size, shape, and spacing of objects when a three-dimensional scene is projected onto a two-dimensional image from a particular viewpoint. In a forward-facing camera image, objects closer to the camera generally appear larger, while objects farther away appear smaller. Parallel lines extending into the distance, such as the boundaries of a straight road, appear to converge towards a point called the vanishing point. For example, a straight road may have parallel lane boundaries in the real world, but in the camera image, the boundaries appear to move closer together as they extend towards the horizon. This is a natural consequence of perspective projection, rather than an actual change in the width of the road.

Why does perspective distortion matter in autonomous vehicles?

Perspective distortion affects how a vehicle interprets road geometry from camera images.

- **Lane Geometry:** Lane boundaries appear closer together in the distance, affecting the estimation of lane width and road geometry.
- **Distance Estimation:** Objects that appear small may be farther away rather than physically small.
- **Path Planning:** Perspective must be considered when interpreting road layout and the positions of surrounding objects.

Question 3: What is a Bird's-Eye View (BEV)? Why can a top-down representation make navigation-related perception easier?
A Bird's-Eye View (BEV) is a representation of a scene from an approximately top-down perspective, as if the scene were being observed from directly above. In autonomous vehicles, a forward-facing camera captures the road in perspective, causing nearby objects to appear larger than distant ones. Lane boundaries may appear to converge towards the horizon, making it difficult to estimate their relative positions directly from the image. A BEV transformation uses a geometric transformation, commonly a perspective transformation, to convert the camera image into an approximate top-down view of the road. This makes the spatial arrangement of road features easier to interpret.
Advantages of BEV for Navigation:

- **Simplified Lane Geometry:** Lane boundaries can appear more parallel in a top-down representation, making their relative positions easier to analyse.

- **Improved Spatial Understanding:** The relative positions of lane markings, road boundaries, and other visible features become easier to interpret.
- **Drivable-Area Estimation:** A top-down view can help estimate the shape and extent of the road region available for driving.
- **Path Planning:** BEV can simplify the estimation of a suitable path relative to lane boundaries and other road features.
- **Reduced Perspective Effects:** Apparent changes in spacing caused by perspective can be reduced, making geometric comparisons easier.

Thank you.